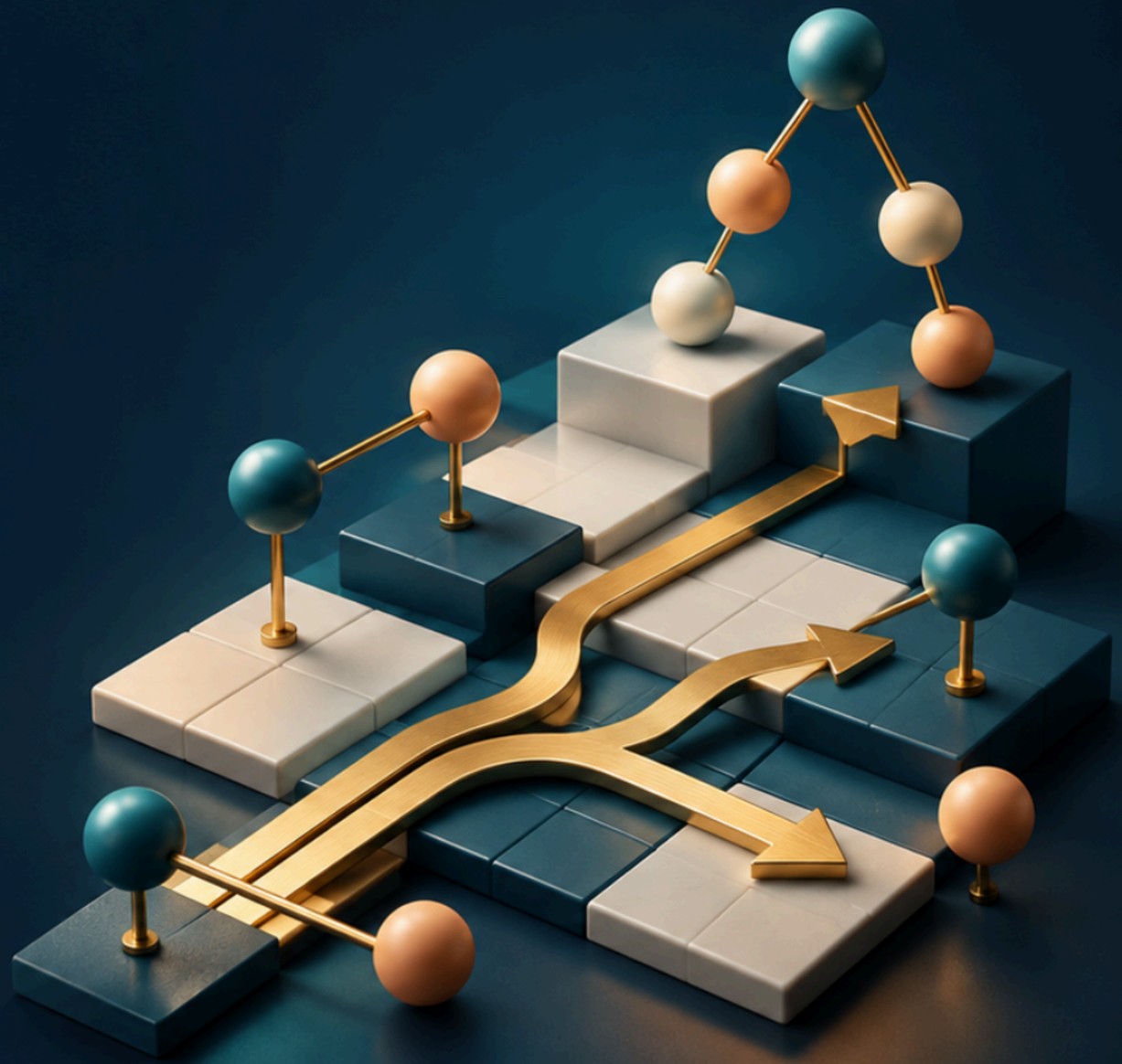
07

백트래킹 기법

하나 골라 보고, 안 맞으면 돌아와
다른 것을 골라 봅니다.

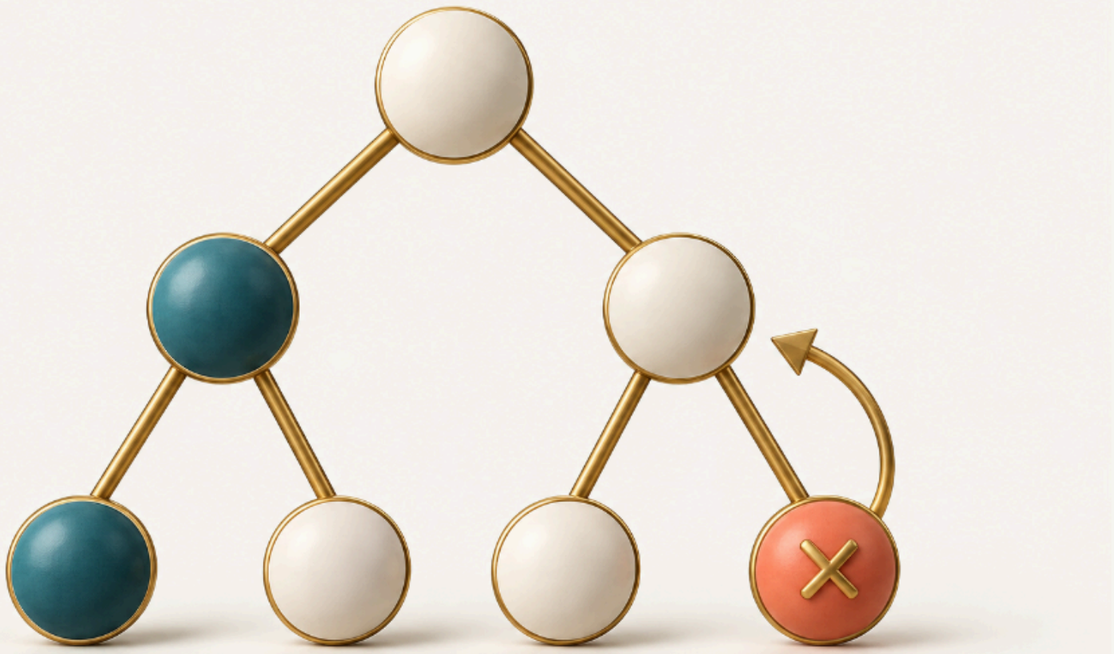


백트래킹

하나 골라 보고, 안 맞으면 돌아와 다른 것을 골라 봅니다.

한 선택을 해 보고 다음 단계로 간다.

조건에 맞지 않으면 이전 단계로 돌아와 다른 선택을 시도한다.



7-1 카드를 순서 있게 놓는 경우의 수

1부터 n까지 적힌 서로 다른 카드 n장이 있다. 이 중 r장을 골라 왼쪽부터 한 줄로 놓는다. 같은 카드를 두 번 쓸 수 없으며, 놓인 순서가 다르면 다른 경우로 센다.

01

가능한 경우의 수를 정수로 반환하시오.

F07-01 n, r

7-1 입력과 반환값

Python: permutation_count(n, r)

n: 카드 수. 0~8의 정수
r: 놓을 카드 수. 0≤r≤n

반환값

배치의 개수인 정수 하나를 반환한다. 배치들을 나열한 목록은 반환하지 않는다.

선택·판정 조건

[1,2]와 [2,1]은 서로 다른 배치이다.
r=0이면 아무것도 놓지 않은 빈 배치 한 가지로 센다.

7-1 두 자리를 채움 · 전부 사용

두 자리를 채움

n=3, r=2

반환값 6

[1,2],[1,3],[2,1],[2,3],[3,1],[3,2]의
여섯 배치이므로 숫자 6을 반환한다.

전부 사용

n=3, r=3

반환값 6

첫 자리 3가지, 다음 2가지, 마지막
1가지로 3×2×1이다.

7-1 한 장만 선택 · 빈 배치

한 장만 선택

`n=4, r=1`

반환값 4

1,2,3,4 중 어느 카드를 놓을지만 고른다.

빈 배치

`n=3, r=0`

반환값 1

놓은 카드가 없어도 빈 배치 한 가지는 존재한다.

7-1 카드도 없음

카드도 없음

`n=0, r=0`

반환값

이 경우에도 빈 배치 하나를 센다.

1

7-1 예제의 계산 과정

첫 카드	둘째 카드 후보	경우 수
1	2 또는 3	2
2	1 또는 3	2
3	1 또는 2	2
합계		6

한 장 놓을 때마다 남은 카드 수가 하나 줄어듭니다.

7-1 한 자리를 채우면 후보가 한 장 줄어든다

카드 세 장으로 두 자리를 채우면 첫 자리에 3가지 선택이 있다. 어떤 카드를 골랐든 둘째 자리에는 남은 2장 중 하나를 놓는다. 따라서 3×2=6이다. 자리가 r개이면 n부터 하나씩 줄어드는 수를 r번 곱한다.

7-1 코드에서 사용할 값

변수	n=3, r=2의 변화
answer=1	곱셈을 시작하는 값
i=0	3-0=3을 곱해 answer=3
i=1	3-1=2를 곱해 answer=6
return answer	경우의 수 6 반환

이 문제는 선택 수를 곱하면 됩니다. 다음 문제부터 선택을 직접 나눠 봅니다.

7-1 Python

```python
def permutation_count(n, r):
    answer = 1
    for i in range(r):
        answer *= n-i
    return answer
```

시간 O(r), 추가 공간 O(1)

7-1 C 함수의 인자

함수 원형

int permutation_count(int n, int r)

배열과 작업 공간

n과 r은 Python과 같은 뜻이다.

7-1 C

```
1  int permutation_count(int n, int r) {
2      int answer=1;
3      for (int i=0; i<r; ++i) answer*=n-i;
4      return answer;
5  }
```

시간 O(r), 추가 공간 O(1)

7-2 합이 목표가 되는 선택의 수

양의 정수 배열 a에서 원소를 몇 개 골라 합을 만든다. 각 위치의 원소는 한 번만 고를 수 있다. 고른 원소의 합이 target과 같은 선택이 몇 개인지 센다.

02

합이 target인 선택의 수를 정수로 반환하시오.

F07-02 a, target

7-2 입력과 반환값

Python: subset_sum_count(a, target)

a: 길이 0~20의 배열. 각 값은 1~50인 정수
target: 목표 합. 0~1000의 정수

반환값

조건을 만족하는 선택의 개수를 반환한다. 가능한지 여부나 선택한 원소의 개수를 반환하지 않는다.

선택·판정 조건

같은 값이라도 위치가 다르면 다른 원소이다.
선택한 원소를 나열하는 순서는 구분하지 않는다. 1과 2를 고른 것과 2와 1을 고른 것은 같은 선택이다.
아무것도 고르지 않은 선택도 포함하며, 그 합은 0이다.

7-2 서로 다른 두 선택 · 값이 같은 두 원소

서로 다른 두 선택

`a=[1,2,3], target=3`

반환값 2

1과 2를 고르거나, 3만 고르는 두 방법이다.

값이 같은 두 원소

`a=[1,1], target=1`

반환값 2

첫 번째 1을 고르는 것과 두 번째 1을 고르는 것은 다르다.

7-2 순서는 세지 않음 · 빈 선택

순서는 세지 않음

`a=[1,2], target=3`

반환값 1

두 원소를 모두 고르는 한 방법이다. 1+2와 2+1로 두 번 세지 않는다.

빈 선택

`a=[], target=0`

반환값 1

아무것도 고르지 않은 한 방법의 합이 0이다.

7-2 만들 수 없는 합

만들 수 없는 합

`a=[2,4], target=3`

반환값

가능한 합은 0,2,4,6이다. 3은 없다.



7-2 예제의 계산 과정

고른 수	합	목표 3과 같나
없음	0	아니요
1	1	아니요
2	2	아니요
3	3	예
1,2	3	예
1,3 또는 2,3 또는 전부	4, 5, 6	아니요

각 원소에서 고르거나 건너뛰는 두 경우를 모두 봅니다.

7-2 원소 하나마다 두 갈래로 나뉜다

첫 원소 1을 보는 순간에는 고르거나 건너뛰는 두 선택이 있다. 1을 골랐다면 다음 원소 2를 볼 때 합은 1이고, 건너뛰었다면 합은 0이다. 이 과정을 마지막 원소까지 반복하고, 마지막 합이 target인 갈래에서만 1을 더한다.

search는 남은 선택을 계산하는 함수입니다

search(i,total)은 i번부터 볼 차례이고 지금까지 합이 total이라는 뜻입니다.

호출	지금까지의 선택	다음에 할 일
search(0,0)	아직 안 고름	**첫 원소 1을 볼 차례**
search(1,0)	1을 건너뜀	**다음 원소 2를 볼 차례**
search(1,1)	1을 고름	**다음 원소 2를 볼 차례**

함수 안에서 자기 함수를 다시 호출하는 것을 재귀라고 부릅니다.

7-2 코드에서 사용할 값

변수	뜻
i	지금 볼 원소 번호. 0이면 첫 원소
total	지금까지 고른 수들의 합
search(i+1,total)	이번 수를 건너뛴 갈래
search(i+1,total+a[i])	이번 수를 고른 갈래

백트래킹은 선택을 나눠 살펴보고, 돌아와 다른 선택도 보는 방법입니다.

7-2 Python

```python
def subset_sum_count(a, target):
    def search(i, total):
        if total > target:
            return 0
        if i == len(a):
            return int(total == target)
        return (search(i+1, total)
                + search(i+1, total+a[i]))
    return search(0, 0)
```

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

7-2 C 함수의 인자

함수 원형

int subset_sum_count(const int a[], int n, int target)

배열과 작업 공간

n은 배열 a의 원소 수이다.

7-2 C

```
1   static int subset_search(const int a[], int n, int target,
2       int i, int total) {
3       if (total>target) return 0;
4       if (i==n) return total==target;
5       return subset_search(a,n,target,i+1,total)
6           + subset_search(a,n,target,i+1,total+a[i]);
7   }
8   int subset_sum_count(const int a[], int n, int target) {
9       return subset_search(a,n,target,0,0);
10  }
```

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

7-3 이웃한 점을 다른 색으로 칠하기

무방향 그래프의 각 점에 색 하나를 칠한다. 선으로 연결된 두 점은 서로 다른 색이어야 한다. 사용할 수 있는 색은 1번부터 k번까지이며, 모든 색을 반드시 사용할 필요는 없다.

03

이 조건으로 모든 점을 칠할 수 있으면 1, 없으면 0을 반환하시오.

F07-03 graph, k

7-3 입력과 반환값

Python: can_color(graph, k)

graph: 점 1~8개의 0/1 연결 표. graph[i][j]=1이면 i와 j가 연결됨
k: 쓸 수 있는 색 수. 1~3의 정수

반환값

가능하면 정수 1, 불가능하면 정수 0이다. 색 배치나 배치의 개수를 반환하지 않는다.

선택·판정 조건

graph는 정사각 표이고 대칭이다. 자기 자신과의 연결은 없어 대각선은 0이다.
연결되지 않은 두 점은 같은 색이어도 된다.
점이 여러 덩어리로 나뉘어 있어도 모든 점을 칠한다.

7-3 삼각형, 두 색 · 삼각형, 세 색

삼각형, 두 색

세 점이 모두 서로 연결된 삼각형, k=2

반환값 0

마지막 점은 앞의 두 점 모두와 이웃하므로 남는 색이 없다.

삼각형, 세 색

같은 삼각형, k=3

반환값 1

세 점에 서로 다른 색을 칠하면 된다.

7-3 떨어진 점도 있음 · 색을 모두 쓸 필요 없음

떨어진 점도 있음

0-1만 연결되고 2는 혼자인 그래프, k=2

반환값 1

0과 1만 다르게 칠하면 된다. 2는 어느 색이나 가능하다.

색을 모두 쓸 필요 없음

점 하나, k=3

반환값 1

한 색만 사용해도 조건을 만족한다.

7-3 연결된 두 점, 한 색

연결된 두 점, 한 색

선 하나로 연결된 두 점, k=1

반환값

양 끝의 색을 다르게 할 수 없다.



7-3 예제의 계산 과정

칠할 점	시도할 색	결과
A	빨강	가능
B	빨강	A와 이웃하므로 제외
B	파랑	가능
C	빨강	A와 이웃하므로 제외
C	파랑	B와 이웃하므로 제외
C	초록	세 색을 쓸 수 있으면 가능

한 점의 색을 고를 때 이미 칠한 이웃만 확인하면 됩니다.

7-3 색을 정하기 전에 이미 칠한 이웃을 본다

삼각형에서 0번을 빨강으로 칠했다면 1번은 빨강을 쓸 수 없다. 1번을 파랑으로 칠한 뒤에는 2번에서 빨강과 파랑이 모두 막힌다. 세 번째 색이 있으면 성공하고, 없으면 앞의 선택으로 돌아간다. 코드의 color[v]=0은 방금 칠한 색을 지우는 줄이다.

연결을 0과 1의 표로 적습니다

삼각형 A,B,C를 코드에서는 0,1,2로 부릅니다.

출발 점	점0	점1	점2
점0	0	1	**1**
점1	1	0	**1**
점2	1	1	**0**

graph[0][2]=1이므로 점0과 점2는 서로 연결되어 있습니다.

7-3 코드에서 사용할 값

코드	뜻
graph[v][u]==1	v와 u가 선으로 연결됨
color[u]==c	이미 칠한 u의 색이 지금 시도하는 색 c와 같음
color[v]=c	현재 점의 색을 c로 선택
color[v]=0	선택을 지우고 다른 색을 시도

인접 행렬은 연결을 적은 표입니다. 행 v·열 u가 1이면 두 점이 이웃입니다.

7-3 Python (1/2)

```python
def can_color(graph, k):
    n = len(graph)
    color = [0]*n
    def paint(v):
        if v == n:
            return 1
        for c in range(1, k+1):
            allowed = True
            for u in range(v):
                if graph[v][u] == 1 and color[u] == c:
                    allowed = False
                    break
```

7-3 Python (2/2)

```
            if allowed:
                color[v] = c
                if paint(v+1) == 1:
                    return 1
                color[v] = 0
        return 0
    return paint(0)
```

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

7-3 C 함수의 인자

함수 원형

int can_color(const int graph[][8], int n, int k)

배열과 작업 공간

graph는 연결 표, n은 정점 수이다. 실제로는 앞의 n행·n열을 사용한다.

7-3 C (1/2)

```
1  static int paint(const int g[][8], int n, int k,
2      int v, int color[]) {
3      if (v==n) return 1;
4      for (int c=1; c<=k; ++c) {
5          int ok=1;
6          for (int u=0; u<v; ++u)
7              if (g[v][u] && color[u]==c) ok=0;
8          if (ok) {
9              color[v]=c;
10             if (paint(g,n,k,v+1,color)) return 1;
11             color[v]=0;
12         }
13     }
14     return 0;
15 }
```

7-3 C (2/2)

```
16  int can_color(const int graph[][8], int n, int k) {
17      int color[8]={0};
18      return paint(graph,n,k,0,color);
19  }
```

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

7-4 서로 공격하지 않는 퀸의 배치 수

n×n 체스판에 퀸 n개를 놓는다. 퀸은 같은 행, 같은 열, 같은 대각선에 있는 다른 퀸을 공격한다. 어느 두 퀸도 서로 공격하지 않도록 놓아야 한다.

04

조건을 만족하는 배치의 수를 정수로 반환하시오.

F07-04 n

7-4 입력과 반환값

Python: nqueen_count(n)

n: 체스판의 한 변 길이이자 퀸 수. 1~8의 정수

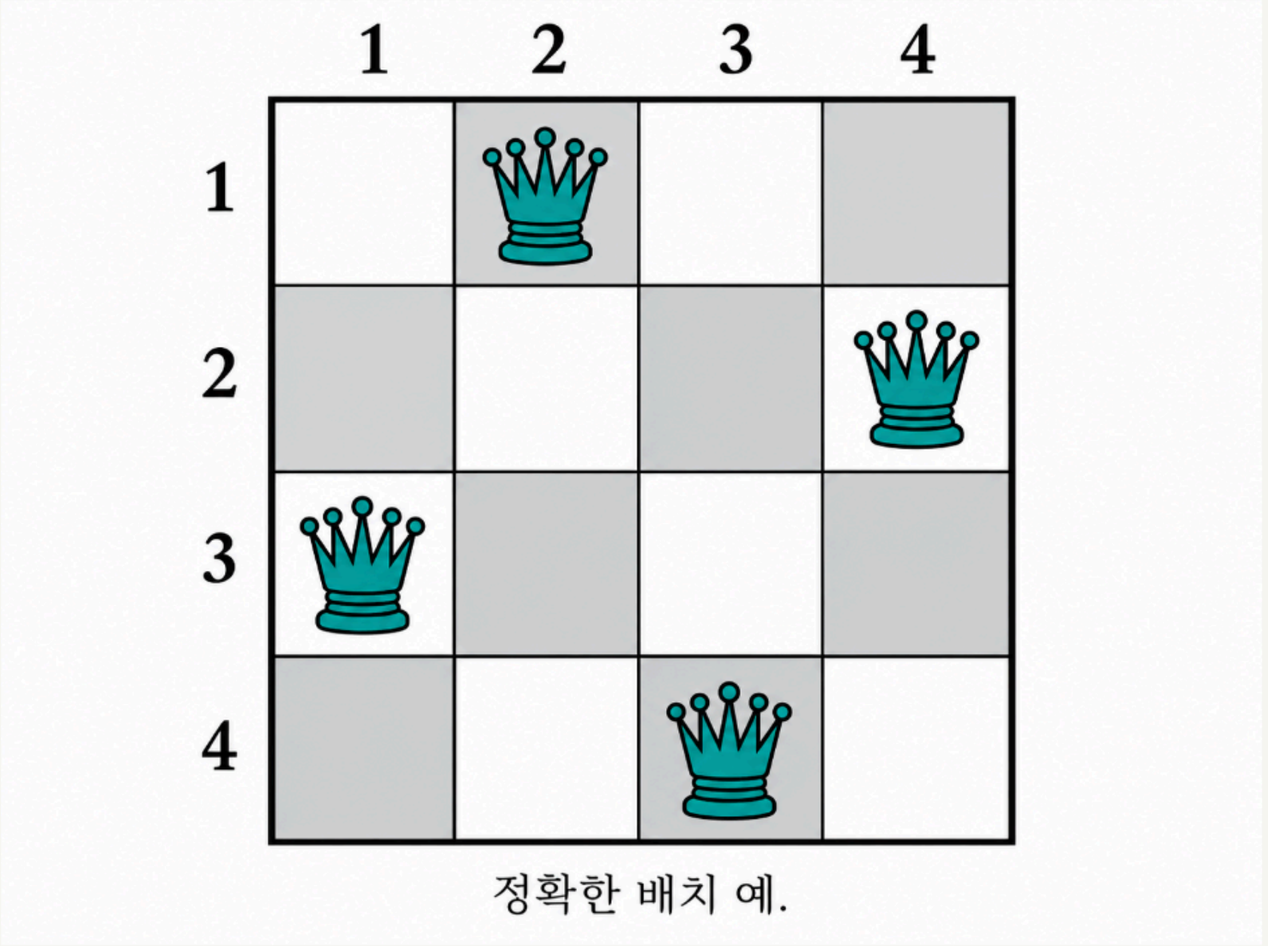
반환값

가능한 배치 수를 반환한다. 가능한 배치가 없으면 0이다.

선택·판정 조건

한 칸에는 퀸 하나만 놓는다.
회전하거나 뒤집은 모양도 퀸이 놓인 칸이 다르면 다른 배치로 센다.
퀸 n개를 n개의 행에 서로 공격하지 않게 놓으면 각 행과 열에 하나씩 놓이게 된다.

7-4 서로 공격하지 않는 퀸의 배치 수



정확한 배치 예.

그림의 행과 열은 1부터 센다. 위에서부터 2,4,1,3열에 놓으면 서로 공격하지 않는다. 코드의 열 번호는 1,3,0,2이다.

7-4 한 칸 · 두 줄

한 칸

n=1

반환값 1

유일한 칸에 퀸 하나를 놓는다.

두 줄

n=2

반환값 0

행과 열을 다르게 놓아도 같은 대각선에 놓이게 된다.

7-4 세 줄 · 네 줄

세 줄

n=3

반환값 0

세 퀸을 모두 충돌 없이 놓는 배치가 없다.

네 줄

n=4

반환값 2

위에서부터 2,4,1,3열인 배치와 그 좌우 대칭 배치를 각각 센다.

7-4 다섯 줄

다섯 줄

n=5

반환값

조건을 만족하는 서로 다른 열 선택이 열 가지이다.

10

7-4 예제의 계산 과정

놓을 행	열	앞 퀸과 충돌 여부
첫째	2	앞 퀸 없음
둘째	4	열 다름, 대각선 아님
셋째	1	앞의 두 퀸과 충돌 없음
넷째	3	앞의 세 퀸과 충돌 없음

한 행에 하나씩 놓으므로 같은 행 여부는 따로 검사하지 않아도 됩니다.

7-4 행 차이와 열 차이가 같으면 대각선이다

(1행,2열)과 (2행,3열)은 행과 열이 모두 한 칸 차이이므로 같은 대각선이다. 코드는 퀸을 한 행에 하나씩 놓아 같은 행 검사를 생략한다. 앞 행의 퀸과 열이 같은지, 행 차이와 열 차이의 절댓값이 같은지를 검사한다.

그림의 열 번호와 코드의 열 번호

그림은 1부터, 배열과 코드는 0부터 셉니다.

그림의 행	그림의 열	코드의 열
1	2	**1**
2	4	**3**
3	1	**0**
4	3	**2**

같은 배치를 코드의 columns에는 [1,3,0,2]로 저장합니다.

7-4 코드에서 사용할 값

코드	뜻
columns.append(col)	현재 행에서 고른 열을 목록에 추가
col != c	앞 퀸의 열 c와 이번 열 col이 달라야 함
abs(col−c) != row−r	열 차이와 행 차이가 달라야 대각선 충돌 없음
columns.pop()	방금 고른 열을 지우고 다른 열도 시도

예를 들어 (1행,2열)과 (2행,3열)은 행·열 차이가 모두 1이어서 같은 대각선입니다.

7-4 Python (1/2)

```python
def nqueen_count(n):
    columns = []
    def place(row):
        if row == n:
            return 1
        answer = 0
        for col in range(n):
            allowed = True
            for r in range(row):
                c = columns[r]
                if col == c or abs(col-c) == row-r:
                    allowed = False
                    break
```

7-4 Python (2/2)

```
14                if allowed:
15                    columns.append(col)
16                    answer += place(row+1)
17                    columns.pop()
18            return answer
19        return place(0)
```

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$

7-4 C 함수의 인자

함수 원형

int nqueen_count(int n)

배열과 작업 공간

n은 한 변의 칸 수이면서 놓을 퀸 수이다.

7-4 C (1/2)

```
1   static int queens(int n, int row, int pos[]) {
2       if (row==n) return 1;
3       int answer=0;
4       for (int col=0; col<n; ++col) {
5           int ok=1;
6           for (int r=0; r<row; ++r) {
7               int d=col-pos[r];
8               if (d<0) d=-d;
9               if (col==pos[r] || d==row-r) ok=0;
10          }
11          if (ok) {
12              pos[row]=col;
13              answer+=queens(n,row+1,pos);
14          }
15      }
```

7-4 C (2/2)

```
16          return answer;
17      }
18      int nqueen_count(int n) {
19          int pos[8]={0};
20          return queens(n,0,pos);
21      }
```

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$